

République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique

Higher Institute of Sciences



Projet de fin d'étude

En : Mathématiques et Informatique

Spécialité : Ingénierie de Systèmes d'Information & Logiciels

Intitulé :

Vectorization Automatique d'Images Matricielles par Deep Learning

Encadré par :

ABDELLAHOUM Hamza

Présenté par :

BITAM Amir

AITHAMOUDI Zakaria

Membres du Jury :

????

????

Soutenu le :

??/07/2026

Année universitaire : 2025/2026

Remerciements

Nous tenons à exprimer notre profonde gratitude envers Allah, le Tout-Puissant, pour nous avoir accordé la force, la patience et la persévérance nécessaires pour mener à bien ce projet de fin d'étude.

Nous remercions sincèrement notre encadreur **ABDELLAHOUAM Hamza** pour sa précieuse guidance, son soutien constant et ses précieux conseils tout au long de ce projet.

Nos vifs remerciements s'adressent également aux membres du jury **????** et **????** pour avoir accepté d'évaluer notre travail.

Dédicace

Résumé

Nous proposons dans le cadre de notre projet de fin d'études un modèle d'IA qui traite le zoom et le dézoom dans les images numériques. En zoomant sur une image de taille donnée, sa qualité est souvent altérée et l'image peut devenir pixélisée. Notre travail consiste à améliorer ces images en augmentant leur qualité.

En ce qui concerne le dézoom, en réduisant les dimensions de l'image, il faut minimiser le nombre de pixels tout en préservant l'esthétique de l'image.

Nous avons créé un modèle CNN avec une architecture adaptée, piloté par des métaheuristiques (algorithme génétique) permettant de sélectionner les paramètres optimaux afin d'obtenir de bonnes performances avec un nombre minimal de paramètres.

Mots-clés : Redimensionnement d'images, Super Résolution, Réseaux de Neurones Convolutifs (CNN), Algorithme Génétique, Métaheuristiques, PSNR, SSIM.

Abstract

Keywords : Image Resizing, Super Resolution, Convolutional Neural Networks (CNN), Genetic Algorithm, Metaheuristics, PSNR, SSIM.

Liste des Acronymes

XML	eXtensible Markup Language
SVG	Scalable Vector Graphics
GA	Genetic Algorithm
CNN	Convolutional Neural Networks
PSNR	Peak Signal to Noise Ratio
HR	High Resolution
LR	Low Resolution
SSIM	Structural SIMilarity
SRCNN	Super Resolution Convolutional Neural Network
SRGAN	Super Resolution Generative Adversarial Network
MAE	Mean Absolute Error
MSE	Mean Squared Error
ReLU	Rectified Linear Unit
ResNet	Residual Network
SE	Squeeze-and-Excitation
CAR	Content Adaptive Resampler
FARF	Feature Augmented Random Forest
DRDN	Deep Residual Dense Network
EDSAN	Enhanced Dense Space Attention Network
PPI	Pixels Per Inch
ViT	Vision Transformer
UDF	Unsigned Distance guided Focal loss

Table des matières

Remerciements	i
Dédicace	ii
Résumé	iii
Abstract	iv
Liste des Acronymes	v
Introduction Générale	2
1 État de l'Art	3
1.1 Introduction	3
1.2 Définitions liées à la vectorisation d'images	3
1.2.1 Images matricielles	3
1.2.1.1 Pixels	4
1.2.2 Images vectorielles	4
1.2.2.1 Courbes de Bézier	5
1.2.2.2 Format SVG	5
1.3 Apprentissage automatique et apprentissage profond	5
1.3.1 Apprentissage automatique et apprentissage auto-supervisé . . .	5
1.3.2 Apprentissage profond	6
1.3.2.1 Réseaux de neurones	7
1.3.2.2 Transformers	7
1.3.2.3 Vision Transformers	7
1.4 Mécanismes d'attention	7
1.4.1 Self-attention	8
1.4.2 Cross-attention	9
1.5 Concepts utilisés dans la vectorisation par apprentissage profond	9
1.5.1 Superpixels	9
1.5.2 Rendu différentiable	10
1.5.3 Fonctions de perte	10
1.5.4 Apprentissage coarse-to-fine	11
1.6 Évaluation des résultats	11
1.6.1 MSE	11
1.6.2 PSNR	12

1.6.3	SSIM	12
1.6.4	LPIPS	13
1.6.5	Temps d'inférence	13
1.7	Travaux connexes	14
1.7.1	Approches algorithmiques	14
1.7.1.1	Potrace	14
1.7.2	Approches basées sur l'optimisation	15
1.7.2.1	DiffVG	15
1.7.2.2	LIVE	15
1.7.3	Approches basées sur l'apprentissage profond	16
1.7.3.1	Im2Vec	16
1.7.3.2	SuperSVG	16
1.8	Synthèse comparative des approches	17
1.9	Problématique	18
1.10	Conclusion	19
2	Approche Proposée	20
2.1	Introduction	20
2.2	Dataset et prétraitement	20
2.2.1	Le prétraitement des données	20
2.2.1.1	Former les paires d'images	20
2.2.1.2	Augmentation de données	20
2.3	Méthode proposée	20
2.3.1	Définir les hyperparamètres évalués par les métaheuristiques	20
2.3.1.1	L'Algorithme Génétique	20
2.3.1.2	Les paramètres à optimiser	20
2.3.1.3	Évaluation d'une solution	20
2.3.2	Architecture	20
2.3.2.1	Sur-échantillonnage (Upsampling)	20
2.3.2.2	Sous-échantillonnage (Downsampling)	20
2.4	Conclusion	20
3	Expérimentations et Résultats	21
3.1	Introduction	21
3.2	Environnement de développement	21
3.2.1	Hardware	21
3.2.2	Software	21
3.3	Dataset utilisé	21
3.4	Prétraitement	21
3.5	Évaluation	21

3.5.1	Évaluation quantitative	21
3.5.2	Fonction de perte	21
3.6	Paramétrage de l'Algorithme Génétique	21
3.7	Sous-échantillonnage	22
3.7.1	Hyperparamètres avec la métaheuristique (GA)	22
3.7.2	Résultats	22
3.7.2.1	Résultats quantitatifs	22
3.7.2.2	Résultats qualitatifs	22
3.8	Sur-échantillonnage	22
3.8.1	Hyperparamètres	22
3.8.2	Résultats	23
3.8.2.1	Résultats quantitatifs	23
3.8.2.2	Résultats qualitatifs	23
3.9	Conclusion	23
Conclusion Générale		24
Bibliographie		25

Table des figures

1.1	Comparaison entre une image matricielle (gauche) sujette à la pixellisation lors de l'agrandissement, et une image vectorielle (droite) conservant sa netteté grâce à sa définition mathématique	4
1.2	Exemple du contenu d'un fichier .svg	5

Liste des tableaux

1.1	Synthèse comparative des principales approches de vectorisation étudiées.	17
3.1	Spécifications de la machine Kaggle	21
3.2	Valeurs du PSNR en fonction du nombre de générations	22
3.3	Résultats du downsampling (Set5) — PSNR / SSIM	22
3.4	Comparaison quantitative sur Set5 (PSNR en dB)	23

Introduction Générale

Le traitement d'images est un domaine de recherche important qui trouve des applications dans divers domaines tels que la médecine, la surveillance, la sécurité, la robotique, l'automobile, etc.

L'un des problèmes les plus courants dans ce domaine est la qualité de l'image qui peut être altérée lorsqu'elle est agrandie ou réduite en taille.

Ce mémoire est organisé comme suit :

- **Chapitre 1** : État de l'art — revue de littérature sur le redimensionnement d'images et les approches existantes.
- **Chapitre 2** : Approche proposée — architecture des modèles CNN et sélection des hyperparamètres par méta heuristiques.
- **Chapitre 3** : Expérimentations et Résultats — évaluation quantitative et qualitative avec comparaisons.

Chapitre 1

État de l'Art

1.1 Introduction

L'intelligence artificielle par le biais de l'apprentissage automatique a imposé un changement de paradigme dans de nombreux domaines[1], en particulier l'informatique et ses différentes disciplines. La vision par ordinateur est un parfait exemple de ce changement de paradigme, où la spécialité s'est dirigé de l'application d'algorithmes "naïf" à l'utilisation de réseaux de neurones pour le traitement et l'extraction d'informations de l'image, cette évolution dans les techniques utilisées par la vision par ordinateur est illustré dans l'évolution de la vectorisation d'images. Dans ce chapitre, nous exposerons d'abord les concepts clés à la vectorisation d'images, à l'apprentissage automatique, et les architectures utilisées tout au long de ce travail, ceci nous permettra de mieux appréhender ensuite les travaux effectués sur le sujet.

1.2 Définitions liées à la vectorisation d'images

Dans cette section nous tenterons de définir les termes et concepts lié à la transformation d'image matricielles en images vectorielles, dite, vectorisation d'images. En soulignant notamment la différence entre les images dites matricielles et vectorielles. En opposant ainsi les deux types d'images nous pourrions mieux cerner la motivation derrière ce travail et pourquoi la vectorisation d'image constitue un exercice complexe de la vision par ordinateur.

1.2.1 Images matricielles

Les images matricielles pourrait être décrites de la même manière que [2] le fait, soit des fonction bidimensionnelles $f(x, y)$, où x et y sont des coordonnées spatiales planes et où f associerait à chaque paire une amplitude ou valeur correspondante, ou plus simplement comme étant une carte ou grille de points de pixels, de largeur et de hauteur donné. Par définition les images matricielles présente une limite lié à l'agrandissement de l'image, Effectivement, plus l'image est agrandi plus la perte de détails s'accroît, la mise à l'échelle est donc impossible par la nature fixe du nombre de pixels constituant l'image.

1.2.1.1 Pixels

Un pixel, contraction de *picture element*, représente l'unité élémentaire d'une image numérique. Il correspond à un point de la grille matricielle et possède une position, définie par ses coordonnées spatiales, ainsi qu'une valeur décrivant son intensité ou sa couleur. Dans une image en niveaux de gris, cette valeur correspond généralement à une intensité lumineuse, tandis que dans une image couleur elle est souvent représentée par plusieurs composantes, par exemple rouge, vert et bleu. Ainsi, une image matricielle peut être comprise comme un ensemble fini de pixels organisés selon une largeur et une hauteur données [2]. Cette représentation rend le contenu directement dépendant de la résolution : lorsqu'une image est fortement agrandie, les pixels deviennent progressivement visibles, ce qui produit l'effet de pixellisation. Ainsi, le pixel constitue l'unité élémentaire de l'image matricielle, mais il impose également une granularité fixe qui limite la conservation des détails lors des changements d'échelle.

1.2.2 Images vectorielles

Les images vectorielles sont décrites par un ensemble de formes géométriques individuelles : (cercles, segments de droites, courbes de Bézier, etc.), chacune de ces formes est enregistrée accompagnée de paramètres mathématiques et attributs la décrivant (couleur de remplissage, point d'encrage, etc.). Contrairement aux images représentées sous forme matricielles où les pixels sont figés, ici les images offrent une possibilité d'agrandissement théoriquement infini, et cela sans dégrader la qualité de l'image en question car le rendu image est calculé de nouveau pour chaque résolution [4]. Les images vectorielles sont donc très utilisées dans le graphisme et la conception, rendant le passage d'image matricielle à une image vectorielle d'autant plus critique pour ces domaines d'application, mais aussi complexe dû à la nature différente des représentations matricielle et vectorielle.

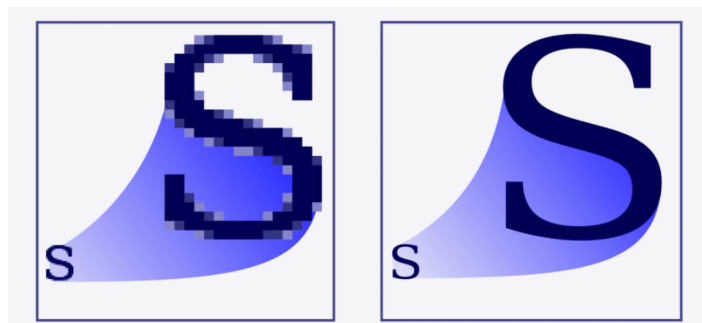


FIGURE 1.1 – Comparaison entre une image matricielle (gauche) sujette à la pixellisation lors de l'agrandissement, et une image vectorielle (droite) conservant sa netteté grâce à sa définition mathématique

1.2.2.1 Courbes de Bézier

Une courbe de Bézier est définie comme une fonction polynomiale dont la forme est dictée par la position de points de contrôle. La courbe passe par le premier et le dernier point (points d'ancrage), tandis que les points intermédiaires agissent comme des "aimants" qui tirent la courbe vers eux. Les courbes de Bézier font partie des formes géométriques individuelles qui composent les images vectorielles et se distinguent par le faible nombre de données numériques nécessaires à la représentation de formes complexes

1.2.2.2 Format SVG

SVG (Scalable Vector Graphics) est un format basé sur XML comportant des directives pour la définition de graphiques 2D[5]. Stockées dans des fichiers textes rendant les SVG compressible et indexable, ces directives SVG supportent aussi bien les formes géométriques vectorielle que les images et le texte. Pour les raisons précédentes ce sera ce format qui sera préféré pour la sortie du modèle.

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<!DOCTYPE svg PUBLIC "-//W3C//DTD SVG 1.1//EN" "http://
www.w3.org/Graphics/SVG/1.1/DTD/svg11.dtd">
<svg width="391" height="391" viewBox="-70.5 -70.5 391 391"
xmlns="http://www.w3.org/2000/svg" xmlns:xlink="http://
www.w3.org/1999/xlink">
<rect fill="#fff" stroke="#000" x="-70" y="-70" width="390"
height="390"/>
<g opacity="0.8">
  <rect x="25" y="25" width="200" height="200" fill="lime"
stroke-width="4" stroke="pink" />
  <circle cx="125" cy="125" r="75" fill="orange" />
  <polyline points="50,150 50,200 200,200 200,100" stroke="red"
stroke-width="4" fill="none" />
  <line x1="50" y1="50" x2="200" y2="200" stroke="blue" stroke-
width="4" />
</g>
</svg>
```

FIGURE 1.2 – Exemple du contenu d'un fichier .svg

1.3 Apprentissage automatique et apprentissage profond

1.3.1 Apprentissage automatique et apprentissage auto-supervisé

L'apprentissage automatique est une discipline du domaine de l'intelligence artificielle dans laquelle, contrairement à l'approche programmatique classique, la machine ne reçoit pas une suite d'instructions détaillées pour résoudre une tâche donnée, car ces instructions seraient souvent trop complexes à définir explicitement [6]. Le principe consiste plutôt à apprendre à partir d'un ensemble de données, afin d'améliorer pro-

gressivement une performance P sur une tâche T à partir d'une expérience E , selon la définition proposée par Mitchell [7]. Selon la nature des données disponibles, on distingue généralement l'apprentissage supervisé, où chaque exemple d'entrée est associé à une cible ou étiquette attendue, et l'apprentissage non supervisé, où le modèle doit découvrir par lui-même des structures présentes dans les données, comme des groupes d'exemples similaires ou des représentations de plus faible dimension [8].

Cependant, dans le contexte de la vision par ordinateur moderne, ces deux catégories ne suffisent pas toujours à expliquer l'intérêt des modèles pré-entraînés utilisés pour l'extraction de caractéristiques visuelles. En effet, l'annotation manuelle de grands ensembles d'images est une tâche coûteuse et parfois difficile à généraliser à tous les domaines. C'est dans ce cadre que l'apprentissage auto-supervisé prend une place importante. Cette approche exploite des données non étiquetées tout en générant automatiquement un signal de supervision à partir des données elles-mêmes, par exemple en demandant au modèle de prédire une partie manquante, une transformation appliquée ou une relation entre différentes vues d'une même image [9]. Elle permet ainsi d'apprendre des représentations visuelles générales à partir de grandes collections d'images sans annotations humaines explicites, représentations qui peuvent ensuite être transférées vers d'autres tâches de vision par ordinateur [6, 9]. Dans le cadre de la vectorisation d'images, cet aspect est particulièrement pertinent, car l'objectif n'est pas seulement de classer une image, mais d'en extraire une représentation riche de sa structure, de ses régions et de ses détails visuels, afin de faciliter ensuite la prédiction de primitives vectorielles.

1.3.2 Apprentissage profond

L'apprentissage profond est une sous-branche de l'apprentissage automatique fondée principalement sur les réseaux de neurones profonds. Ces modèles sont constitués de plusieurs couches de neurones artificiels capables d'apprendre progressivement des représentations de plus en plus abstraites à partir des données. Cette approche s'est fortement développée grâce à l'augmentation de la puissance de calcul, à la disponibilité de grandes quantités de données et aux progrès réalisés dans l'entraînement des réseaux profonds [6]. Dans le domaine du traitement d'images, les réseaux convolutionnels CNN ont longtemps constitué l'une des architectures dominantes, grâce à leur capacité à exploiter la structure spatiale locale des images et à extraire automatiquement des caractéristiques visuelles telles que les contours, les formes ou les textures [2]. Cependant, les architectures récentes fondées sur l'attention, en particulier les Transformers et les Vision Transformers, ont progressivement élargi cette approche en permettant de modéliser des relations plus globales entre les différentes régions d'une image.

1.3.2.1 Réseaux de neurones

Les réseaux de neurones artificiels sont des modèles d'apprentissage automatique inspirés, de manière simplifiée, du fonctionnement des neurones biologiques. Ils sont constitués d'unités de calcul appelées neurones artificiels, organisées en couches et reliées entre elles par des connexions pondérées. L'apprentissage consiste à ajuster ces poids à partir des données d'entraînement afin que le réseau puisse produire une sortie correcte pour une tâche donnée [6]. Dans le domaine du traitement d'images, ces modèles sont utilisés pour apprendre automatiquement des fonctions de décision à partir d'exemples, notamment dans les problèmes de classification, de reconnaissance de formes et de reconstruction [2].

1.3.2.2 Transformers

Les Transformers sont une architecture de réseaux de neurones introduite initialement pour le traitement des séquences, notamment en traduction automatique. Contrairement aux réseaux récurrents, ils ne traitent pas les éléments de la séquence de manière strictement séquentielle, mais s'appuient principalement sur des mécanismes d'attention, en particulier l'auto-attention, afin de modéliser les relations entre les différents éléments d'une entrée [10]. Grâce à sa capacité à apprendre des dépendances complexes et à être entraînée efficacement sur de grandes quantités de données, cette architecture a ensuite été largement utilisée dans de nombreux domaines de l'apprentissage profond, notamment le traitement du langage naturel, la vision par ordinateur et les modèles multimodaux [6].

1.3.2.3 Vision Transformers

Les Vision Transformers, ou ViT, sont une adaptation de l'architecture Transformer au traitement des images. Au lieu de s'appuyer principalement sur des convolutions comme les CNN, une image est divisée en petits blocs appelés patches, par exemple de taille 16×16 . Ces patches sont ensuite transformés en vecteurs et traités comme une séquence d'éléments, de manière similaire aux tokens dans le traitement du langage naturel [11]. Grâce au mécanisme d'auto-attention, le modèle peut apprendre des relations globales entre différentes régions de l'image. Cette capacité rend les Vision Transformers particulièrement intéressants pour les tâches où la compréhension de la structure globale de l'image est importante, en complément des informations locales.

1.4 Mécanismes d'attention

Les mécanismes d'attention constituent une composante importante des architectures modernes d'apprentissage profond, en particulier dans les modèles de type Transformer.

Leur objectif est de permettre au modèle d'accorder une importance différente aux éléments d'une entrée selon leur pertinence pour la tâche à résoudre, au lieu de traiter toutes les informations de manière uniforme. Ce fonctionnement repose généralement sur trois éléments principaux : une requête, des clés et des valeurs. La requête représente l'information recherchée, les clés servent à mesurer la similarité avec cette requête, et les valeurs contiennent les informations à combiner.

Dans le cas de l'attention par produit scalaire mise à l'échelle, les requêtes, les clés et les valeurs sont regroupées respectivement dans les matrices Q , K et V . Les scores d'attention sont obtenus en calculant le produit entre les requêtes et les clés, puis en le normalisant par $\sqrt{d_k}$, où d_k désigne la dimension des clés. Une fonction softmax est ensuite appliquée afin d'obtenir des poids d'attention, utilisés pour pondérer les valeurs. Ce mécanisme est formalisé par l'équation suivante [10] :

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V \quad (1.1)$$

Ainsi, la sortie est obtenue comme une combinaison pondérée des valeurs, ce qui permet au modèle de se concentrer davantage sur les éléments les plus utiles. De manière intuitive, l'attention peut être vue comme un mécanisme de récupération d'information différentiable, où le modèle apprend à retrouver les éléments importants d'une séquence ou d'une représentation en fonction du contexte [6]. Cette idée est particulièrement utile pour modéliser des dépendances à longue distance, que ce soit dans une séquence de mots ou dans une image découpée en régions ou en patchs. Dans les Transformers, les mécanismes d'attention remplacent en grande partie les traitements séquentiels classiques, ce qui facilite la parallélisation de l'apprentissage et permet de mieux exploiter de grandes quantités de données [10].

1.4.1 Self-attention

La self-attention, ou auto-attention, est un cas particulier du mécanisme d'attention dans lequel les requêtes, les clés et les valeurs proviennent de la même séquence. Elle permet donc à chaque élément d'une entrée de prendre en compte les autres éléments de cette même entrée afin de construire une représentation plus riche et contextualisée. Dans une phrase, par exemple, chaque mot peut être relié aux autres mots pour mieux comprendre son sens selon le contexte. Dans une image représentée sous forme de patchs, chaque patch peut de la même manière interagir avec les autres régions de l'image [10]. Cette capacité à modéliser les relations internes d'une séquence explique l'importance de la self-attention dans les architectures Transformer. Dans le traitement d'images, cette propriété permet notamment de mettre en relation des régions éloignées d'une même image, contrairement aux opérations strictement locales qui se limitent à un voisinage restreint.

1.4.2 Cross-attention

La cross-attention, ou attention croisée, est une variante du mécanisme d'attention dans laquelle les requêtes ne proviennent pas de la même source que les clés et les valeurs. Contrairement à la self-attention, où un élément interagit avec les autres éléments de la même séquence, la cross-attention permet à une représentation de s'appuyer sur une autre représentation externe. Dans un Transformer encodeur-décodeur, par exemple, le décodeur produit les requêtes, tandis que les clés et les valeurs proviennent des représentations calculées par l'encodeur. Cela permet au décodeur de sélectionner les informations importantes de l'entrée lors de la génération de la sortie [6]. Dans le contexte de la vectorisation d'images, ce type d'attention peut être utilisé pour faire interagir des requêtes associées aux primitives vectorielles à générer avec des clés et des valeurs issues des caractéristiques extraites de l'image. Les paramètres vectoriels prédits peuvent ainsi être guidés par les informations visuelles les plus pertinentes, ce qui facilite la génération de primitives adaptées au contenu de l'entrée.

1.5 Concepts utilisés dans la vectorisation par apprentissage profond

La vectorisation d'images par apprentissage profond repose sur plusieurs concepts techniques permettant de passer d'une image matricielle, composée de pixels, à une représentation vectorielle décrite par des formes paramétriques. Contrairement aux méthodes classiques de traçage, les approches récentes cherchent à apprendre ou à optimiser automatiquement les paramètres des primitives vectorielles afin de reconstruire au mieux l'image d'entrée. Pour cela, elles utilisent notamment la décomposition de l'image en régions plus simples, le rendu différentiable pour relier les paramètres vectoriels à leur rendu matriciel, ainsi que des fonctions de perte permettant de mesurer l'écart entre l'image originale et l'image reconstruite. Certaines méthodes adoptent également une stratégie progressive de type *coarse-to-fine*, dans laquelle la structure globale de l'image est d'abord reconstruite avant d'ajouter les détails fins [12, 3].

1.5.1 Superpixels

Les superpixels sont des régions obtenues en regroupant des pixels voisins présentant des caractéristiques similaires, notamment en termes de couleur et de position spatiale. Ils permettent de remplacer une représentation pixel par pixel par une représentation plus compacte, composée de régions visuellement cohérentes. Parmi les méthodes les plus connues, l'algorithme SLIC (*Simple Linear Iterative Clustering*) adapte le principe du k-means afin de générer efficacement des superpixels. Cette méthode est appréciée pour

sa simplicité, sa rapidité, sa faible consommation mémoire et sa capacité à produire des régions qui respectent bien les contours de l'image [13]. Dans le cadre de la vectorisation d'images, les superpixels sont utiles car ils divisent une image complexe en sous-régions plus simples à traiter. SuperSVG exploite précisément cette idée en décomposant l'image d'entrée en superpixels contenant des pixels de couleurs et de textures similaires, puis en vectorisant ces régions de manière progressive [3].

1.5.2 Rendu différentiable

Le rendu différentiable est une technique qui permet de relier une représentation vectorielle à son rendu matriciel tout en conservant la possibilité de calculer des gradients. Dans le contexte de la vectorisation d'images, les paramètres des primitives vectorielles, comme les positions des points de contrôle, les couleurs ou l'épaisseur des traits, sont d'abord rendus sous forme d'image raster. L'image obtenue est ensuite comparée à l'image cible à l'aide d'une fonction de perte, puis les gradients de cette perte sont rétropropagés vers les paramètres vectoriels afin de les optimiser [12]. Cette approche permet donc d'utiliser des méthodes d'optimisation ou d'apprentissage profond pour ajuster automatiquement les formes vectorielles à partir d'une supervision dans le domaine des pixels. Elle est notamment utilisée dans des méthodes comme LIVE, qui optimise progressivement des chemins de Bézier dans un cadre de rendu entièrement différentiable afin de produire des SVG organisés en couches [14].

1.5.3 Fonctions de perte

Les fonctions de perte jouent un rôle central dans la vectorisation d'images par apprentissage profond, car elles permettent de mesurer l'écart entre l'image matricielle d'origine et l'image reconstruite à partir des primitives vectorielles. Dans les approches utilisant un rendu différentiable, les chemins SVG prédits ou optimisés sont d'abord rasterisés, puis comparés à l'image cible à l'aide d'une fonction de perte calculée dans l'espace des pixels. Une perte simple, comme la MSE, peut être utilisée pour minimiser la différence entre les deux images, mais elle ne suffit pas toujours à préserver correctement la structure, les contours ou la couleur des objets. Par exemple, LIVE montre qu'une optimisation fondée uniquement sur la MSE peut conduire à un effet de couleur moyenne, puis propose une perte UDF afin de donner plus d'importance aux pixels proches des contours, ainsi qu'une perte Xing pour limiter les auto-intersections des courbes [14]. Dans SuperSVG, les auteurs combinent plusieurs pertes adaptées à la vectorisation par superpixels, notamment une perte de reconstruction normalisée, une perte de frontière pour empêcher les chemins de dépasser les limites du superpixel, et une perte Dynamic Path Warping permettant au modèle de raffinement d'hériter des connaissances du modèle grossier [3].

1.5.4 Apprentissage coarse-to-fine

L'apprentissage *coarse-to-fine* désigne une stratégie progressive dans laquelle le modèle commence par reconstruire les éléments globaux ou principaux d'une image, avant d'ajouter progressivement des détails plus fins. Dans le contexte de la vectorisation d'images, cette approche est particulièrement utile car une image complexe est difficile à convertir directement en une représentation SVG précise. En procédant par étapes, le modèle peut d'abord approximer la structure générale, puis améliorer localement la reconstruction. SuperSVG adopte cette logique à travers un cadre d'apprentissage en deux étapes : un modèle grossier prédit d'abord des chemins SVG pour reconstruire la structure principale du superpixel, puis un modèle de raffinement ajoute d'autres chemins afin d'enrichir les détails, en étant guidé par les chemins prédits à l'étape précédente [3]. Une idée similaire apparaît dans LIVE, où l'image est reconstruite progressivement en ajoutant et en optimisant de nouveaux chemins de Bézier dans une représentation en couches, ce qui permet d'obtenir un SVG plus compact et plus cohérent avec la structure visuelle de l'image [14].

1.6 Évaluation des résultats

L'évaluation des résultats est une étape essentielle pour mesurer la qualité d'une méthode de vectorisation d'images. Dans ce contexte, l'image SVG générée est généralement rendue sous forme matricielle, puis comparée à l'image d'entrée afin d'estimer la fidélité de reconstruction. Plusieurs métriques complémentaires peuvent être utilisées, car une seule mesure ne suffit pas toujours à juger correctement la qualité visuelle d'un résultat. Les métriques comme la MSE et le PSNR évaluent principalement l'écart pixel par pixel entre l'image originale et l'image reconstruite, tandis que la SSIM cherche davantage à mesurer la similarité structurelle. De son côté, la LPIPS évalue une distance perceptuelle plus proche de la perception humaine. Enfin, le temps d'inférence permet d'évaluer l'efficacité pratique de la méthode, notamment lorsqu'il s'agit de traiter des images complexes ou un grand nombre d'images [3].

1.6.1 MSE

La MSE (*Mean Squared Error*), ou erreur quadratique moyenne, est une métrique utilisée pour mesurer l'écart pixel par pixel entre une image de référence et une image reconstruite. Dans le cas de la vectorisation, l'image SVG générée est d'abord rendue sous forme matricielle, puis comparée à l'image d'entrée. La MSE calcule la moyenne

des carrés des différences entre les valeurs des pixels des deux images :

$$MSE = \frac{1}{HWC} \sum_{i=1}^H \sum_{j=1}^W \sum_{c=1}^C (I_{i,j,c} - \hat{I}_{i,j,c})^2$$

où I représente l'image originale, $\hat{I}$ l'image reconstruite, H et W les dimensions de l'image, et C le nombre de canaux. Une valeur faible de MSE indique donc une meilleure fidélité de reconstruction. Dans les travaux de vectorisation d'images, cette métrique est souvent utilisée pour quantifier la fidélité de reconstruction entre l'image cible et le rendu matriciel obtenu à partir du SVG généré. Elle constitue donc un indicateur simple et directement interprétable de l'erreur pixel, utile pour comparer différents résultats de vectorisation ou pour guider certaines étapes d'optimisation [14, 3]. Cependant, la MSE reste une mesure purement numérique et ne reflète pas toujours parfaitement la qualité visuelle perçue. Dans LIVE, les auteurs montrent par exemple qu'une optimisation basée uniquement sur la MSE peut conduire à un effet de couleur moyenne, car tous les pixels sont pris en compte de la même manière, même ceux qui ne sont pas directement liés à la forme en cours d'optimisation [14].

1.6.2 PSNR

Le PSNR (*Peak Signal-to-Noise Ratio*) est une métrique dérivée de la MSE qui permet d'évaluer la qualité de reconstruction d'une image. Il mesure le rapport entre la valeur maximale possible d'un pixel et l'erreur introduite par la reconstruction. Dans le cas d'images normalisées entre 0 et 1, il peut être exprimé par :

$$PSNR = 10 \log_{10} \left(\frac{1}{MSE} \right)$$

De manière générale, une valeur élevée de PSNR indique que l'image reconstruite est proche de l'image originale, tandis qu'une valeur faible traduit une erreur de reconstruction plus importante. Dans le contexte de la vectorisation, le PSNR est calculé après avoir rendu l'image SVG sous forme matricielle, puis en comparant ce rendu à l'image d'entrée. Il est utilisé dans plusieurs travaux, notamment DiffVG et SuperSVG, afin de mesurer la fidélité de reconstruction entre l'image cible et le rendu SVG obtenu. Cela confirme son intérêt comme indicateur quantitatif pour comparer les résultats de vectorisation, en particulier lorsqu'il s'agit d'évaluer l'erreur de reconstruction au niveau des pixels [12, 3].

1.6.3 SSIM

La SSIM (*Structural Similarity Index Measure*) est une métrique d'évaluation de qualité d'image qui mesure la similarité structurelle entre une image de référence et

une image reconstruite. Contrairement à la MSE et au PSNR, qui se concentrent principalement sur les différences pixel par pixel, la SSIM cherche à comparer des informations plus proches de la perception visuelle humaine, notamment la luminance, le contraste et la structure locale de l'image [15]. Elle peut être exprimée sous la forme :

$$SSIM(x, y) = \frac{(2\mu_x\mu_y + C_1)(2\sigma_{xy} + C_2)}{(\mu_x^2 + \mu_y^2 + C_1)(\sigma_x^2 + \sigma_y^2 + C_2)}$$

où x et y représentent deux fenêtres locales des images comparées, μ_x et μ_y leurs moyennes, σ_x^2 et σ_y^2 leurs variances, σ_{xy} leur covariance, et C_1 , C_2 des constantes de stabilisation. Une valeur élevée de SSIM indique une meilleure préservation de la structure visuelle. Dans le contexte de la vectorisation, SuperSVG utilise la SSIM pour mesurer la distance structurelle entre l'image d'entrée et le rendu obtenu à partir du SVG généré [3].

1.6.4 LPIPS

La LPIPS (*Learned Perceptual Image Patch Similarity*) est une métrique perceptuelle qui mesure la similarité entre deux images à partir de caractéristiques extraites par des réseaux de neurones profonds. Contrairement à la MSE et au PSNR, qui comparent directement les valeurs des pixels, la LPIPS compare les représentations internes obtenues à différents niveaux d'un réseau pré-entraîné. Cette approche permet de mieux prendre en compte la perception visuelle humaine, car deux images peuvent avoir une faible différence pixel par pixel tout en étant visuellement différentes, ou inversement [16]. De manière simplifiée, la LPIPS calcule une distance entre les activations normalisées de deux images :

$$LPIPS(x, \hat{x}) = \sum_l \frac{1}{H_l W_l} \sum_{h,w} \|w_l \odot (\phi_l(x)_{h,w} - \phi_l(\hat{x})_{h,w})\|_2^2$$

où ϕ_l représente les caractéristiques extraites à la couche l , w_l des poids appris, et $\hat{x}$ l'image reconstruite. Une valeur faible de LPIPS indique donc une meilleure similarité perceptuelle. Dans le contexte de la vectorisation, SuperSVG utilise la LPIPS pour évaluer la distance perceptuelle entre l'image d'entrée et le rendu obtenu à partir du SVG généré [3].

1.6.5 Temps d'inférence

Le temps d'inférence désigne le temps nécessaire à un modèle pour produire un résultat à partir d'une image d'entrée. Dans le contexte de la vectorisation d'images, il correspond au temps requis pour générer les chemins SVG ou pour optimiser les paramètres vectoriels représentant l'image. Cette métrique est importante car une

méthode peut obtenir une bonne qualité de reconstruction tout en restant difficilement exploitable si son temps de calcul est trop élevé. Les méthodes fondées sur l'optimisation directe des paramètres SVG peuvent produire des résultats précis, mais elles nécessitent souvent plusieurs itérations de rendu et de rétropropagation, ce qui augmente fortement le temps de traitement. À l'inverse, les méthodes fondées sur l'apprentissage profond cherchent à prédire directement les paramètres vectoriels, ce qui permet généralement une génération plus rapide. Dans SuperSVG, les auteurs évaluent ainsi le temps d'exécution en plus des métriques de qualité comme la MSE, le PSNR, la LPIPS et la SSIM. Leurs résultats montrent que SuperSVG-B atteint un temps d'inférence très court, par exemple 0,31 seconde pour 500 chemins et 0,71 seconde pour 2000 chemins, tandis que certaines méthodes d'optimisation comme LIVE nécessitent un temps beaucoup plus important [3].

1.7 Travaux connexes

La vectorisation automatique d'images a connu une évolution progressive, allant des méthodes algorithmiques classiques jusqu'aux approches récentes combinant apprentissage profond et rendu différentiable. Pour comparer ces travaux, il ne suffit pas de regarder uniquement la similarité visuelle entre l'image d'entrée et le SVG obtenu. Une méthode de vectorisation doit aussi être évaluée selon plusieurs critères, notamment la fidélité visuelle, le temps de calcul, le nombre de chemins générés, la simplicité du SVG, son éditabilité, sa capacité de généralisation et le niveau de contrôle laissé à l'utilisateur [17]. Dans cette section, nous présentons donc les méthodes les plus pertinentes pour notre sujet, en montrant progressivement comment les limites des approches classiques et d'optimisation ont mené vers des méthodes plus récentes comme SuperSVG.

1.7.1 Approches algorithmiques

Les approches algorithmiques reposent généralement sur l'analyse géométrique de l'image, comme l'extraction de contours, la détection de régions ou l'ajustement de courbes. Elles sont souvent rapides et ne nécessitent pas de données d'entraînement, mais restent limitées lorsque l'image contient des couleurs complexes, des textures ou un grand nombre de détails.

1.7.1.1 Potrace

Potrace est une méthode classique de vectorisation principalement conçue pour convertir des images binaires en contours vectoriels [18]. Son principe consiste à extraire les frontières entre les zones noires et blanches, à les approximer par des polygones, puis à transformer ces polygones en courbes de Bézier lisses. Cette méthode est simple,

rapide et efficace pour des images comme les logos, les textes ou les dessins en noir et blanc. Cependant, elle dépend fortement de la qualité de la binarisation et reste peu adaptée aux images complexes, colorées ou texturées. Potrace constitue donc une référence importante pour les approches classiques, mais montre aussi les limites des méthodes purement algorithmiques face à une vectorisation plus générale.

1.7.2 Approches basées sur l'optimisation

Les approches basées sur l'optimisation cherchent à ajuster directement les paramètres des primitives vectorielles afin que le rendu obtenu soit le plus proche possible de l'image d'entrée. Avec le rendu différentiable, il devient possible de calculer des gradients entre l'image rendue et l'image cible, puis de les utiliser pour modifier les courbes, les couleurs ou l'opacité des chemins SVG.

1.7.2.1 DiffVG

DiffVG propose un rasteriseur différentiable permettant de relier le domaine vectoriel au domaine matriciel [12]. Grâce à ce mécanisme, un ensemble de courbes ou de formes vectorielles peut être rendu sous forme d'image, comparé à une image cible, puis optimisé par rétropropagation. Cette approche est importante car elle permet d'utiliser une supervision uniquement matricielle, sans avoir besoin d'un SVG de référence. Elle est donc devenue une base technique pour plusieurs méthodes récentes. Cependant, DiffVG reste coûteux lorsqu'il faut optimiser beaucoup de chemins ou effectuer un grand nombre d'itérations. De plus, le nombre de primitives doit souvent être fixé à l'avance, ce qui limite le contrôle automatique de la complexité du SVG.

1.7.2.2 LIVE

LIVE, pour *Layer-wise Image Vectorization*, améliore les approches d'optimisation en introduisant une reconstruction progressive par couches [14]. La méthode ajoute progressivement des chemins de Bézier, puis les optimise afin de reconstruire l'image de manière *coarse-to-fine*. Cette stratégie permet d'obtenir des SVG plus structurés, plus compacts et plus éditables que ceux produits par une optimisation naïve. LIVE introduit également des pertes spécifiques pour mieux préserver les contours et limiter les auto-intersections des chemins. Malgré ces avantages, la méthode reste lente, car chaque image nécessite une optimisation itérative. Le résultat dépend aussi du nombre de chemins choisi et de plusieurs hyperparamètres. LIVE montre donc l'intérêt d'une représentation vectorielle progressive et éditable, mais confirme aussi le besoin de méthodes plus rapides.

1.7.3 Approches basées sur l'apprentissage profond

Les approches basées sur l'apprentissage profond cherchent à apprendre directement la relation entre une image matricielle et une représentation vectorielle. Contrairement aux méthodes d'optimisation qui ajustent les chemins pour chaque image, ces méthodes entraînent un modèle capable de prédire les paramètres vectoriels à partir de l'image d'entrée, ce qui permet généralement une inférence plus rapide.

1.7.3.1 Im2Vec

Im2Vec propose une méthode de génération de graphiques vectoriels sans supervision vectorielle directe [19]. Le modèle apprend à partir d'images matricielles seulement : il encode l'image dans un espace latent, génère des formes vectorielles, puis utilise un rendu différentiable pour comparer le résultat à l'image cible. Cette approche est intéressante car elle évite le besoin de jeux de données SVG annotés, qui sont difficiles à obtenir et souvent ambigus, puisqu'une même image peut être représentée par plusieurs structures vectorielles différentes. Cependant, Im2Vec reste surtout adapté à des images relativement simples ou appartenant à des domaines limités, comme les icônes, les caractères ou les emojis. Il montre donc l'intérêt de l'apprentissage profond pour la vectorisation, mais reste limité pour traiter des images complexes et variées.

1.7.3.2 SuperSVG

SuperSVG est la méthode la plus proche de notre thématique et constitue une référence importante dans les travaux récents de vectorisation image-to-SVG [3]. La méthode décompose d'abord l'image en superpixels, afin de travailler sur des régions localement plus homogènes en couleur et en texture. Elle utilise ensuite une architecture d'apprentissage profond pour prédire les chemins SVG de chaque région, avec une reconstruction en deux étapes : une étape grossière pour préserver la structure principale, puis une étape de raffinement pour enrichir les détails. SuperSVG combine ainsi plusieurs idées importantes des travaux précédents : le rendu différentiable hérité de DiffVG, la reconstruction progressive proche de LIVE, et la prédiction de chemins SVG par apprentissage profond. Cette combinaison lui permet d'obtenir un meilleur compromis entre qualité de reconstruction et temps de calcul. Néanmoins, ses résultats dépendent encore de la qualité de la décomposition en superpixels et du nombre de chemins utilisés. Malgré ces limites, SuperSVG représente l'approche la plus pertinente pour notre état de l'art, car elle résume l'évolution des méthodes de vectorisation vers des solutions plus rapides, plus précises et mieux adaptées aux images complexes.

1.8 Synthèse comparative des approches

Les travaux étudiés montrent que la vectorisation automatique d'images repose sur un compromis entre plusieurs critères : la fidélité visuelle, le temps de calcul, la simplicité du SVG généré, le nombre de chemins, l'éditabilité et la capacité de généralisation. Les méthodes algorithmiques privilégient généralement la rapidité et la simplicité, mais restent limitées aux images peu complexes. Les méthodes basées sur l'optimisation offrent une meilleure flexibilité grâce au rendu différentiable, mais leur coût augmente avec le nombre de chemins et d'itérations. Les approches basées sur l'apprentissage profond cherchent à dépasser cette limite en apprenant à prédire directement les paramètres vectoriels.

TABLE 1.1 – Synthèse comparative des principales approches de vectorisation étudiées.

Méthode	Idée principale	Points forts	Limites	Rôle dans l'évolution du domaine
Potrace	Extraction de contours à partir d'une image binaire, puis approximation par courbes de Bézier.	Simple, rapide, sans apprentissage, efficace pour les logos, textes et dessins binaires.	Peu adaptée aux images complexes, colorées ou texturées. Dépend fortement de la binarisation.	Représente les approches algorithmiques classiques.
DiffVG	Rasteriseur différentiable permettant d'optimiser les paramètres vectoriels à partir d'une perte dans l'espace image.	Permet la rétropropagation vers les paramètres SVG et ne nécessite pas de SVG de référence.	Optimisation coûteuse, nombre de chemins souvent fixé à l'avance, sensibilité à l'initialisation.	Introduit le rendu différentiable comme outil central pour les méthodes modernes.
LIVE	Optimisation progressive de chemins de Bézier dans une logique couche par couche.	Produit des SVG plus structurés, compacts et éditables.	Temps de calcul élevé et forte dépendance au nombre de chemins et aux hyperparamètres.	Montre l'intérêt d'une reconstruction progressive et organisée.

Méthode	Idée principale	Points forts	Limites	Rôle dans l'évolution du domaine
Im2Vec	Génération de formes vectorielles à partir d'une supervision uniquement matricielle.	Évite la supervision SVG directe et apprend une représentation latente des formes.	Plus adapté aux images simples ou à des domaines limités. Moins efficace pour les images complexes.	Montre l'intérêt de l'apprentissage profond pour prédire des primitives vectorielles.
SuperSVG	Décomposition en superpixels, prédiction de chemins SVG par réseau profond et reconstruction <i>coarse-to-fine</i> .	Combine apprentissage profond, superpixels, rendu différentiable et raffinement progressif. Bon compromis entre qualité et vitesse.	Dépend de la décomposition en superpixels et du nombre de chemins utilisés.	Représente l'approche la plus proche des méthodes récentes de vectorisation image-to-SVG.

Cette comparaison montre que SuperSVG constitue l'approche la plus complète parmi les travaux étudiés. Elle reprend le rendu différentiable introduit par DiffVG, la logique progressive mise en avant par LIVE, et l'efficacité des modèles d'apprentissage profond. Cependant, la vectorisation automatique reste confrontée à plusieurs difficultés, notamment le contrôle de la complexité du SVG, la fidélité visuelle, le temps de calcul et la généralisation à des images variées.

1.9 Problématique

La vectorisation automatique d'images matricielles en SVG reste un problème difficile, car elle doit produire une représentation à la fois fidèle, compacte, éditable et générée dans un temps raisonnable. Une reconstruction visuellement correcte ne suffit pas si le SVG obtenu contient trop de chemins, s'il est difficile à modifier ou si sa génération nécessite une optimisation trop coûteuse.

Les travaux étudiés montrent que les approches existantes répondent partiellement à ce problème. Les méthodes classiques sont rapides mais limitées aux images simples. Les méthodes d'optimisation offrent une meilleure qualité de reconstruction, mais au prix d'un temps de calcul élevé. Les méthodes basées sur l'apprentissage profond améliorent l'efficacité de la génération, mais doivent encore gérer la complexité des images réelles et la structure des SVG produits.

La problématique de ce travail peut donc être formulée ainsi : comment améliorer la vectorisation automatique d'images matricielles en SVG afin d'obtenir une représentation fidèle, compacte et exploitable, tout en limitant le temps de calcul et la complexité de la représentation générée ?

L'enjeu principal est donc de trouver un meilleur compromis entre qualité visuelle, simplicité du SVG, rapidité de génération et capacité de généralisation.

1.10 Conclusion

Dans ce chapitre, nous avons présenté les notions essentielles liées à la vectorisation d'images, puis étudié les principales familles de méthodes existantes. L'analyse des travaux connexes montre une évolution progressive des approches algorithmiques vers les méthodes d'optimisation différentiable, puis vers les modèles basés sur l'apprentissage profond.

Cette évolution met en évidence un compromis central : produire un SVG fidèle à l'image d'entrée tout en conservant une représentation simple, éditable et générée efficacement. Les méthodes récentes comme SuperSVG montrent des avancées importantes dans cette direction, mais la vectorisation automatique reste un problème ouvert lorsque l'on cherche à traiter des images complexes tout en maîtrisant le temps de calcul et la complexité du SVG obtenu.

La suite du mémoire présentera l'approche proposée pour répondre à cette problématique, ainsi que les choix de conception retenus pour la mise en place du système de vectorisation.

Chapitre 2

Approche Proposée

2.1 Introduction

2.2 Dataset et prétraitement

2.2.1 Le prétraitement des données

2.2.1.1 Former les paires d'images

2.2.1.2 Augmentation de données

2.3 Méthode proposée

2.3.1 Définir les hyperparamètres évalués par les métaheuristiques

2.3.1.1 L'Algorithme Génétique

2.3.1.2 Les paramètres à optimiser

2.3.1.3 Évaluation d'une solution

2.3.2 Architecture

2.3.2.1 Sur-échantillonnage (Upsampling)

2.3.2.2 Sous-échantillonnage (Downsampling)

2.4 Conclusion

Chapitre 3

Expérimentations et Résultats

3.1 Introduction

3.2 Environnement de développement

3.2.1 Hardware

TABLE 3.1 – Spécifications de la machine Kaggle

CPU	Mémoire	GPU
1,5 GHz	13 Go RAM + 73,1 Go HDD	Nvidia P100 — 16 Go VRAM

3.2.2 Software

3.3 Dataset utilisé

3.4 Prétraitement

3.5 Évaluation

3.5.1 Évaluation quantitative

3.5.2 Fonction de perte

$$\text{MAE} = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i| \quad (3.1)$$

3.6 Paramétrage de l’Algorithme Génétique

- Taille de population : 10
- Nombre de générations : 6
- Probabilité de mutation : 0,5
- Fonction objective : PSNR

3.7 Sous-échantillonnage

3.7.1 Hyperparamètres avec la métaheuristique (GA)

TABLE 3.2 – Valeurs du PSNR en fonction du nombre de générations

Générations	PSNR (dB)
3	50,079
6	50,219

- Batch size : 19 Kernel size : 3×3
- Epochs : 27 Nombre de filtres : 64
- Nombre de couches : 5
- Learning rate : Adam Optimizer ($10^{-4} \rightarrow 10^{-5}$, limite 5000)

3.7.2 Résultats

3.7.2.1 Résultats quantitatifs

TABLE 3.3 – Résultats du downsampling (Set5) — PSNR / SSIM

Dataset	Facteur	Notre Modèle	Bicubique
Set5	$\times 2$	45,61 / 0,99	36,17 / 0,97
	$\times 3$	44,80 / 0,99	30,35 / 0,94
	$\times 4$	33,85 / 0,97	26,80 / 0,91

3.7.2.2 Résultats qualitatifs

3.8 Sur-échantillonnage

3.8.1 Hyperparamètres

- Batch size : 16 Kernel size : 3×3
- Epochs : 250 Nombre de filtres : 128
- Nombre de blocs résiduels : 12
- Learning rate : Adam Optimizer ($10^{-4} \rightarrow 10^{-5}$, limite 5000)

3.8.2 Résultats

3.8.2.1 Résultats quantitatifs

TABLE 3.4 – Comparaison quantitative sur Set5 (PSNR en dB)

Test set	Facteur	Bicubic	SRGAN	DRDN	EDSAN	CAR	SRCNN	Notre Modèle
Set5	×2	33,66	–	32,95	35,25	34,38	36,66	36,71
	×3	30,39	–	29,18	31,48	–	32,75	32,20
	×4	28,42	29,40	27,35	29,80	28,42	30,49	31,498

3.8.2.2 Résultats qualitatifs

3.9 Conclusion

Conclusion Générale

Au cours de ce projet de fin d'étude, nous avons exploré divers aspects de l'apprentissage profond et de la vision par ordinateur pour résoudre le problème du redimensionnement des images.

Bibliographie

- [1] Vasant DHAR. “The Paradigm Shifts in Artificial Intelligence”. In : *Communications of the ACM* 67.11 (nov. 2024), p. 50-59. DOI : 10.1145/3664804.
- [2] Rafael C. GONZALEZ et Richard E. WOODS. *Digital Image Processing*. 4th. Pearson, 2018. ISBN : 978-1-292-22304-9.
- [3] Teng HU et al. “SuperSVG : Superpixel-based Scalable Vector Graphics Synthesis”. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2024, p. 24892-24901.
- [4] James D. FOLEY et al. *Computer Graphics : Principles and Practice*. 2nd. Addison-Wesley Professional, 1996, p. 8.
- [5] W3C. *Scalable Vector Graphics (SVG) 1.1 (Second Edition)*. W3C Recommendation. World Wide Web Consortium, 2011. URL : <https://www.w3.org/TR/SVG11/>.
- [6] Aurélien GÉRON. *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*. 3rd. O'Reilly Media, 2022. ISBN : 978-1-098-12597-4.
- [7] Tom M. MITCHELL. *Machine Learning*. McGraw-Hill, 1997, p. 2.
- [8] Christopher M. BISHOP. *Pattern Recognition and Machine Learning*. Springer, 2006, p. 3.
- [9] Longlong JING et Yingli TIAN. “Self-Supervised Visual Feature Learning with Deep Neural Networks : A Survey”. In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* 43.11 (2021), p. 4037-4058. DOI : 10.1109/TPAMI.2020.2992393.
- [10] Ashish VASWANI et al. “Attention Is All You Need”. In : *Advances in Neural Information Processing Systems*. T. 30. Curran Associates, Inc., 2017.
- [11] Alexey DOSOVITSKIY et al. “An Image Is Worth 16x16 Words : Transformers for Image Recognition at Scale”. In : *International Conference on Learning Representations*. 2021. URL : <https://arxiv.org/abs/2010.11929>.
- [12] Tzu-Mao LI et al. “Differentiable Vector Graphics Rasterization for Editing and Learning”. In : *ACM Transactions on Graphics* 39.6 (2020), 193 :1-193 :15. DOI : 10.1145/3414685.3417871.
- [13] Radhakrishna ACHANTA et al. “SLIC Superpixels Compared to State-of-the-Art Superpixel Methods”. In : *IEEE Transactions on Pattern Analysis and Machine Intelligence* 34.11 (2012), p. 2274-2282. DOI : 10.1109/TPAMI.2012.120.

- [14] Xu MA et al. “Towards Layer-wise Image Vectorization”. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2022, p. 16314-16323. DOI : 10.1109/CVPR52688.2022.01583.
- [15] Zhou WANG et al. “Image Quality Assessment : From Error Visibility to Structural Similarity”. In : *IEEE Transactions on Image Processing* 13.4 (2004), p. 600-612. DOI : 10.1109/TIP.2003.819861.
- [16] Richard ZHANG et al. “The Unreasonable Effectiveness of Deep Features as a Perceptual Metric”. In : *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. 2018, p. 586-595. DOI : 10.1109/CVPR.2018.00068.
- [17] Maria DZIUBA et al. *Image Vectorization : A Review*. 2023. arXiv : 2306.06441 [cs.CV].
- [18] Peter SELINGER. *Potrace : A Polygon-Based Tracing Algorithm*. Technical report. Sept. 2003. URL : <https://potrace.sourceforge.net/potrace.pdf>.
- [19] Pradyumna REDDY et al. “Im2Vec : Synthesizing Vector Graphics without Vector Supervision”. In : *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 2021, p. 7342-7351.